

Humanoid Football-Kicking Robot

Team 11 Members:

- Ahmed Gaber – 23012033
- Nada Saad - 23011581
- Habiba Ali – 23011063
- Moaaz Salama – 23012085
- Ahmed Ragab – 23011022

CHAPTER 1: INTRODUCTION

Problem statement: Affordable humanoid robots that can perform precise physical tasks (like kicking and walking) remain complex and expensive. Our project addresses this at a low cost while still being educational.

Aims & objectives: Build a miniature robot that

- (1) detects a football via computer vision
- (2) computes joint angles via kinematics
- (3) physically executes a kick using servos

All on a 1000 EGP budget.

CHAPTER 2: LITERATURE REVIEW

Design & Mechanical: A low-cost mini humanoid robot called BERT was designed using accessible components such as Arduino, micro servos, and a servo driver, with a simple and robust mechanical design intended for easy operation. Its design consists of 5 SG90 micro servos on each leg, giving it a height of 28 cm and width of 14 cm. This is very close to our own design and a great direct comparison. [\[Link\]](#)

Electronics & Software:

- **Kinematics:** This leg model driven by SG90-type servos and an ESP32 uses inverse kinematics to calculate joint angles from a desired (X, Y) foot position relative to the hip rather than hard-coding servo angles. This is essentially the same approach that we are using [\[link\]](#)
- **Ball Detection:** ball detection in soccer robots uses a laptop connected to a camera over USB, with detection results processed using the YOLO library and the detected coordinates used to control the robot [\[link\]](#)
- **Kicking motion control:** Common approaches to implementing kicking motion in humanoid robots are based on key-frame techniques, which are inflexible and require significant time to adjust the robot's position relative to the ball. This is a limitation our system avoids by using IK-computed angles. [\[link\]](#)

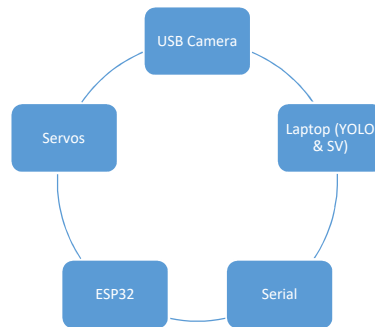
Limitations of Prior Work:

- (1) **Cost & complexity:** Most research-grade humanoid robots (ASIMO, NAO, full-scale pneumatic kickers) are either very expensive or require complex control frameworks like ROS.
- (2) **Balance during kicking:** Accomplishing dynamic kicking motion in a humanoid robot simultaneously requires transferring high kinetic energy to the kicking leg, maintaining balance and stability of the entire body, and managing the impact disturbance from the ball during the kicking moment.
- (3) **Real-time vision complexity:** Many prior systems require real-time continuous video streaming, which introduces latency and computational overhead.
- (4) **SG90 servo limitations:** The SG90 micro servo has a drawback because it has only one axis of rotation, limiting the range of motion per joint.

Humanoid Football-Kicking Robot

CHAPTER 3: PROPOSED SYSTEM

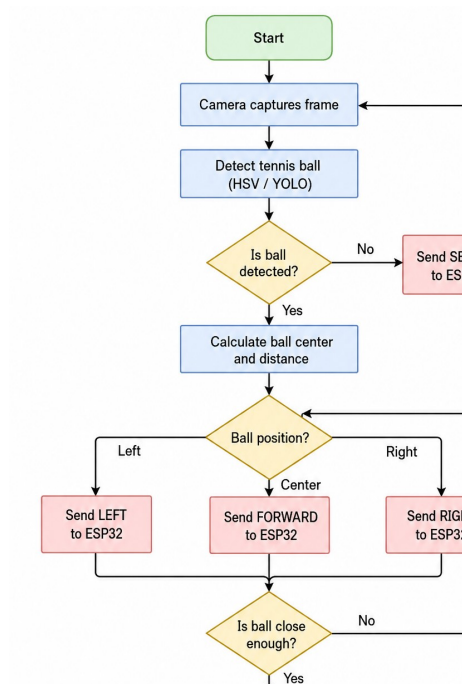
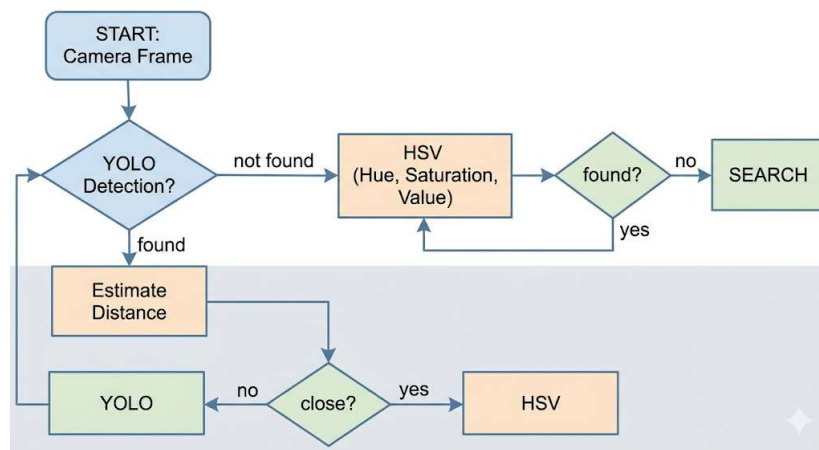
System overview:



Improvements over existing work:

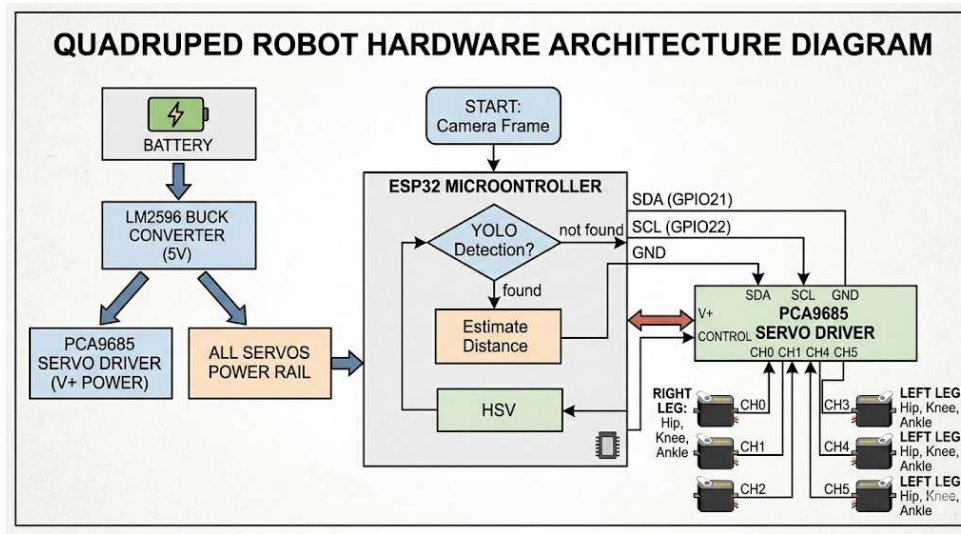
- Lower cost
- simpler architecture (no ROS, no Raspberry Pi)
- stationary ball assumption eliminates need for real-time streaming
- 2D planar IK is sufficient for the task.

Software System Block Diagram:



Humanoid Football-Kicking Robot

Hardware System Block Diagram:

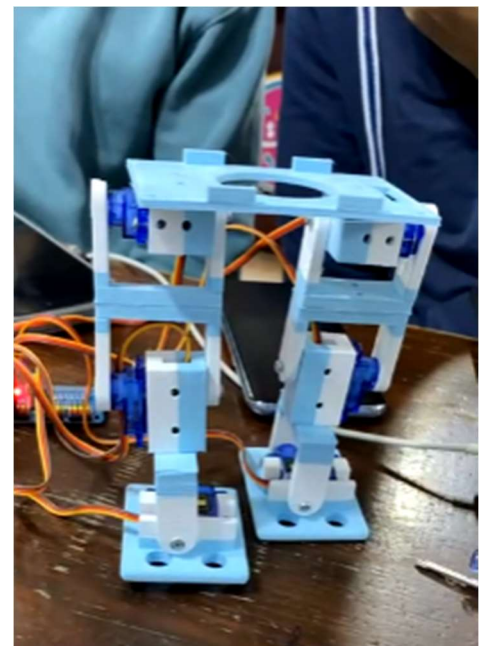


CHAPTER 4: IMPLEMENTATION

Mechanical design: Three joints per kicking leg (hip + knee + feet), fixed base for stability. The non-kicking leg is fixed during kick execution to serve as a stable support base. All PWM signals are generated by a PCA9685 driver module connected to the ESP32 which offloads timing precision from the microcontroller.

State Machine:

- **IDLE:** Waiting for a serial command. This is the startup state.
- **SEARCH:** The robot performs a slow side-to-side sweep motion to scan for the ball. Entered when Python sends SEARCH (no ball detected).
- **ALIGN:** The robot adjusts its orientation. Entered when Python sends LEFT or RIGHT, indicating the ball is off-center.
- **APPROACH:** The robot walks forward toward the ball. Entered when Python sends FORWARD (ball is centered but not close enough).
- **KICK:** The robot executes the kicking motion sequence. The hip extends forward, knee snaps through, then returns to standing. Entered when Python sends KICK (ball is close enough).



Software Implementation:

The Python script runs on the laptop and handles all perception and decision logic. It is structured in six sequential sections:

- **Camera setup:** OpenCV sees the ball using a Cable Web Cam.
- **YOLO + HSV detection:** Each camera frame is first checked using YOLO object detection. If the ball is not confidently detected, the frame is converted from BGR to HSV color space to isolate the tennis ball's yellow-green color. Morphological opening is applied to reduce noise.
- **Contour detection:** OpenCV detects contours in the HSV mask and the largest valid contour is selected as the tennis ball candidate while small false detections are filtered out.
- **Ball position and distance estimation:** A bounding circle is fitted around the detected contour or YOLO bounding box to obtain the ball centre coordinates and radius r . These values are then used to estimate the ball's position and distance from the camera.

Humanoid Football-Kicking Robot

- **Distance estimation** — Using a pre-calibrated pixels-per-cm ratio (measured once by placing the ball at a known distance), the pixel radius is converted to an estimated real-world distance in centimeters.
- **Decision logic and serial communication** — Based on the estimated distance, the script sends one of five ASCII commands over PySerial: *SEARCH*, *LEFT*, *RIGHT*, *FORWARD*, or *KICK*. The command is a short string terminated with a newline, e.g. "*KICK*\n".

Component	Qty	Unit Cost (EGP)	Total (EGP)
ESP32 (owned)	1	-	0
SG90 servo motor	4	80	320
Printing	1	350	670
PCA9685 PWM driver	1	190	860
Converter	1	35	890
Others (Batter pack, wires, breadboard etc.)	1	80	970

CHAPTER 5: RESULTS

Inverse:

```
PS C:\Users\Ahmed Kazak\Downloads\Embedded> & "C:
ed Kazak/Downloads/Embedded/kinematics/leg_kinema
Target: (4, -8)
Angles: (-105.2452637187006, 83.62062979155719)
FK Position: (4.000000000000001, -8.0)
Error cm: 8.881784197001252e-16
```

Vision: There is approximately 300 ms of delay, the detection accuracy for the ball is from 82% to 94%

CHAPTER 6: CONCLUSION

All core project objectives were met: the robot successfully detected the ball, navigated toward it, and executed the kick within the two-week development timeline and is relatively cheap

Challenges Encountered:

- **Servo precision limits:** The SG90 is a low-cost hobby servo with a positional accuracy of roughly $\pm 2-3$ degrees. At the scale of the printed leg links, this translates to a foot positioning error of several millimeters, which was sufficient for the kicking task but would be a limiting factor in any application requiring accuracy
- **Pixel-to-centimeter calibration:** The conversion from detected pixel radius to real-world distance assumed a fixed camera position and a consistent ambient lighting condition. In practice, minor changes in placement or lighting affected the calibration ratio
- **Serial communication timing:** Because the Python script sends a command and the ESP32 executes a motion sequence that takes several hundred milliseconds, care was needed to avoid sending a new command before the previous motion completed. This was handled with a fixed delay on the Python side after each transmission